

Heart Disease Prediction System

Project Id: AIML- 18

Name : VARTIKA GOSAI (Team Leader)

Class Roll Number: 75

University Roll Number : 2028904

Section and Group: K G2

Name : PARIDHI CHOUDHARY

Class Roll Number: 60

University Roll Number : 2028791

Section and Group: K G2

Name : NANDINI RANA

Class Roll Number: 56

University Roll Number : 2028778

Section and Group: K G2

1. Problem Description	
2. Architecture	
3. Models model.....	
4. Evaluation metrics used	
5. Preliminary Results and Visualizations	
6. Conclusion and Future Scope.....	
7. References	

1. Problem Description

Heart disease is one of the leading causes of mortality worldwide. Early and accurate prediction of heart disease can enable timely medical intervention and significantly reduce fatality rates. Manual diagnosis based on clinical parameters is time-consuming and prone to human error, underscoring the need for an automated, data-driven approach.

The Heart Disease Prediction System is a Python-based machine learning application designed to automate the analysis and prediction of heart disease risk in patients. The system leverages core machine learning concepts including data preprocessing, feature selection, multiple classification algorithms, model evaluation, and result visualization to identify at-risk individuals efficiently and accurately.

1.1 Dataset Description

The dataset consists of 10,485 patient entries and 14 columns (13 features + 1 target). The columns are described below:

Column	Feature Name	Description	Values / Range
1	age	Patient Age	29 – 77 years
2	sex	Gender	0 = Female, 1 = Male
3	cp	Chest Pain Type	1=Typical, 2=Atypical, 3=Non-anginal, 4=Asymptomatic
4	trestbps	Resting Blood Pressure (mmHg)	94 – 200
5	chol	Serum Cholesterol (mg/dl)	126 – 564
6	fbs	Fasting Blood Sugar > 120 mg/dl	0 = No, 1 = Yes
7	restecg	Resting ECG Results	0=Normal, 1=ST-T Abnormality, 2=LV Hypertrophy
8	thalach	Max Heart Rate Achieved	71 – 202
9	exang	Exercise-Induced Angina	0 = No, 1 = Yes
10	oldpeak	ST Depression (Exercise vs Rest)	0.0 – 6.2
11	slope	Slope of Peak Exercise ST Segment	1=Upsloping, 2=Flat, 3=Downsloping

12	ca	Major Vessels by Fluoroscopy	0 – 3
13	thal	Thalassemia Type	3=Normal, 6=Fixed Defect, 7=Reversible Defect
14	target	Heart Disease Diagnosis (Target)	0 = No Disease, 1 = Heart Disease

2. Modules for Project Implementation

2.1 Modules Used

pandas

Pandas is used for data loading, manipulation, exploration, and preprocessing. It provides efficient data structures for structured data operations.

- `pd.read_csv()` – Reads the heart disease CSV dataset into a DataFrame
- `df.info()` – Displays dataset structure, column types, and non-null counts
- `df.describe()` – Generates descriptive statistics for all numeric columns
- `df.isnull().sum()` – Identifies missing values in each column
- `df.drop_duplicates()` – Removes duplicate records from the dataset
- `df.hist()` – Plots histograms of all features for distribution analysis
- `df.corr()` – Computes correlation matrix between all numeric features
- `pd.get_dummies()` – Performs one-hot encoding on categorical columns
- `pd.DataFrame()` – Creates DataFrames from dictionaries for new patient input

numpy

NumPy is used for numerical computations, array manipulations, and mathematical operations.

- `np.sqrt()` – Used to compute the initial value of K in KNN as $\sqrt{N}$
- `np.array` operations – Underlying array computation for ML model inputs

matplotlib.pyplot

Matplotlib is the primary plotting library used for generating charts and graphs.

- `plt.figure()` – Creates a new figure for plotting
- `plt.plot()` – Draws line plots for KNN and Decision Tree accuracy curves
- `plt.bar()` – Creates bar charts for SVM kernel and model comparison
- `plt.xlabel()`, `plt.ylabel()`, `plt.title()` – Adds labels and titles to plots
- `plt.xticks()`, `plt.tight_layout()`, `plt.show()` – Formats and displays plots

seaborn

Seaborn is used for enhanced statistical visualizations, particularly the correlation heatmap.

- `sns.heatmap()` – Renders an annotated heatmap of the feature correlation matrix using the 'Purples' colormap

scikit-learn (sklearn)

Scikit-learn is the core machine learning library providing preprocessing, model training, selection, and evaluation tools.

- StandardScaler – Standardises continuous features by removing mean and scaling to unit variance
- train_test_split – Splits dataset into training (70%) and testing (30%) subsets with stratification
- KNeighborsClassifier – Implements KNN classification with configurable K value
- LogisticRegression – Binary classifier using the logistic (sigmoid) function
- SVC – Support Vector Classifier with multiple kernel options (linear, poly, rbf, sigmoid)
- DecisionTreeClassifier – Tree-based classifier with configurable max_features and max_depth
- RandomForestClassifier – Ensemble of decision trees using bagging (n_estimators=100)
- SequentialFeatureSelector – Forward sequential feature selection using Random Forest estimator
- accuracy_score, precision_score, recall_score, f1_score – Performance evaluation metrics
- classification_report – Comprehensive per-class precision, recall, F1-score table
- confusion_matrix – Computes the confusion matrix for model predictions

pickle

Pickle is used for model serialisation and deserialisation, enabling the best-trained model, scaler, and feature list to be saved and reloaded for future predictions without retraining.

- pickle.dump() – Serialises and saves the model, scaler, and selected features to .pkl files
- pickle.load() – Loads saved model, scaler, and features back into memory

warnings

The warnings module is used to suppress non-critical warning messages during model training and library operations, ensuring cleaner output.

- warnings.filterwarnings('ignore') – Suppresses all warning messages

Platform Used

Platform / Tool	Description and Purpose
Python 3.x	The primary programming language for implementing all machine learning models, data preprocessing pipelines, and visualizations.
Jupyter Notebook	An interactive web-based environment for writing, running, and documenting code cell-by-cell. Ideal for exploratory data analysis, model building, and inline visualization.
Visual Studio Code	A lightweight but powerful code editor with Python extensions, debugger, and integrated terminal, used for code writing, testing, and debugging.

scikit-learn

Industry-standard Python ML library providing implementations of KNN, Logistic Regression, SVM, Decision Tree, Random Forest, feature selection, and evaluation metrics.

3. Machine Learning Models / Techniques Used

3.1 K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a non-parametric, instance-based learning algorithm. For a given test point, it computes the distance (typically Euclidean) to all training samples and assigns the class label of the majority among the K nearest neighbors.

In this project, K was varied from 1 to $\sqrt{N}$ (where N is the training set size). For each K, accuracy was computed on the test set. The optimal K was selected as the one achieving the highest accuracy.

3.2 Logistic Regression

Logistic Regression is a linear classifier that models the probability of a binary outcome. It applies the sigmoid function to a linear combination of input features to output a probability between 0 and 1.

The model was trained with `max_iter=1000` and `random_state=42`. It achieved an accuracy of 65.42% on the 70:30 test split.

3.3 Support Vector Machine (SVM)

SVM finds the optimal hyperplane that maximises the margin between the two classes in the feature space. For non-linearly separable data, the kernel trick maps inputs to higher dimensions.

Four kernels were tested: linear, polynomial (poly), Radial Basis Function (rbf), and sigmoid. The RBF kernel achieved the highest accuracy of 65.29%, and was selected as the best SVM model.

3.4 Decision Tree

Decision Trees partition the feature space by recursively splitting on the most informative features according to criteria such as Gini impurity or Information Gain. Each leaf node represents a class label.

In this project, `max_depth` was fixed at 5 to prevent overfitting. The `max_features` parameter was varied from 1 to the total number of selected features. The optimal `max_features` value was 4, achieving an accuracy of 65.67%.

3.5 Random Forest

Random Forest is an ensemble learning method that builds multiple decision trees during training (bagging) and outputs the majority vote across all trees. It reduces overfitting compared to a single tree and improves generalisation.

The model used 100 estimators (`n_estimators=100`) and `random_state=42`. It achieved an accuracy of 65.26% on the test set.

Feature Selection

Sequential Feature Selector (SFS) with forward selection was applied using a Random Forest estimator with 5-fold cross-validation. From the full encoded feature set, the 8 most informative features were selected to reduce dimensionality and improve model generalisability.

4. Evaluation Metrics Used

All models were evaluated on the held-out test set (30% of the data) using the following metrics:

Accuracy

Accuracy measures the overall percentage of correct predictions made by the model across all classes.

Precision (Positive Predictive Value) measures the proportion of predicted positive cases that are actually positive. High precision minimises false alarms

Recall (Sensitivity)

Recall measures the proportion of actual positive cases that were correctly identified by the model. High recall minimises missed disease cases — critical in medical diagnosis.

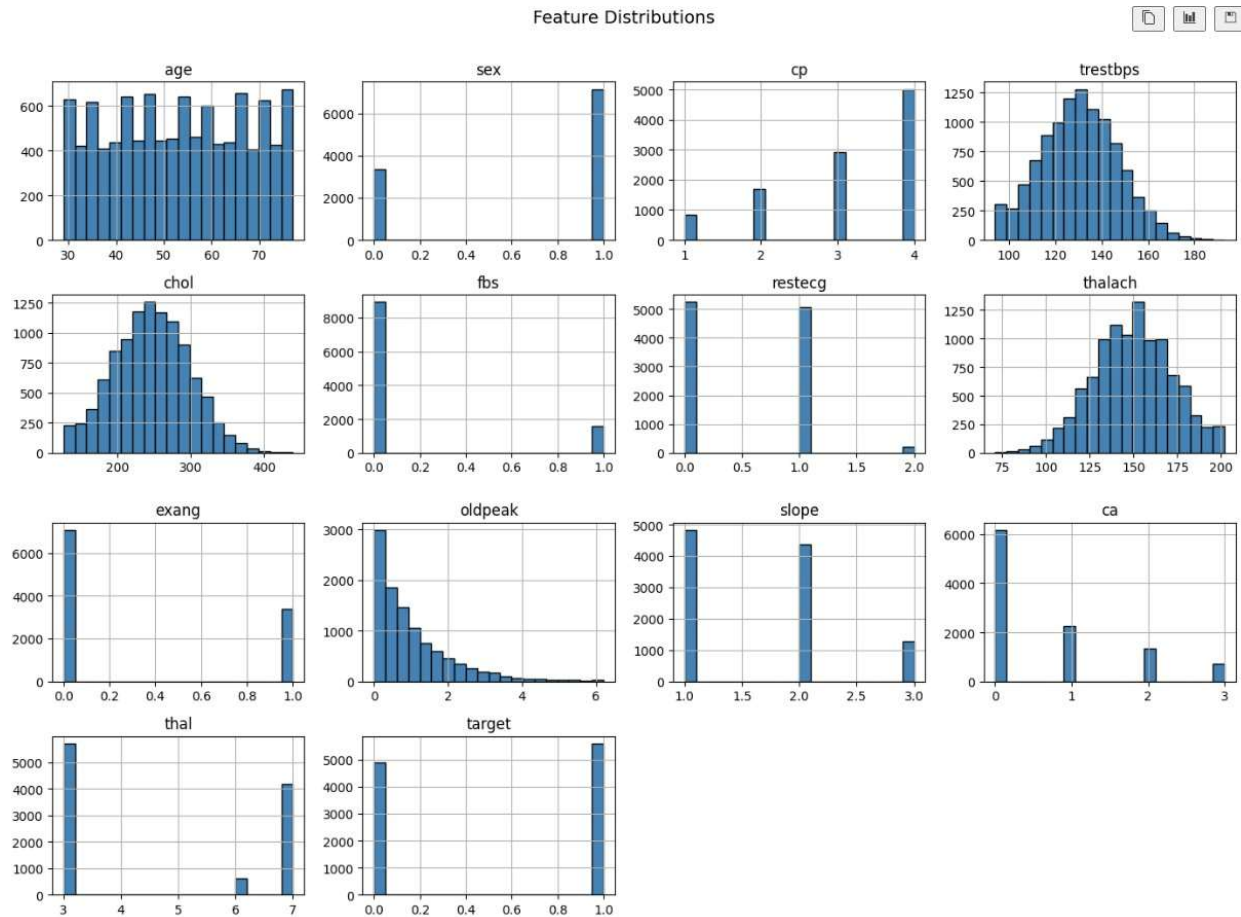
The F1 Score is the harmonic mean of Precision and Recall, providing a balanced metric especially useful when class distribution is slightly imbalanced

Confusion Matrix

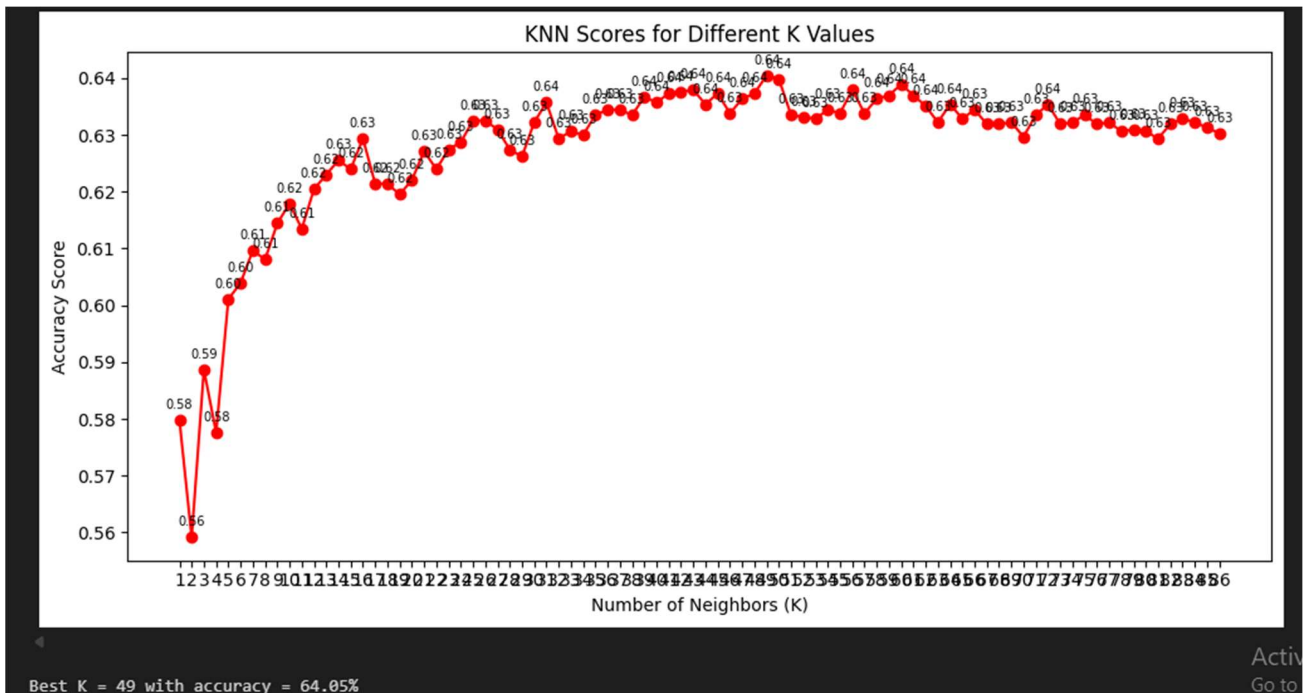
The confusion matrix provides a tabular summary of prediction results broken down into True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN), offering a detailed view of classification errors.

5.Results and Visualizations

Histograms were plotted for all 14 features to understand the distribution, spread, and frequency of each clinical attribute in the dataset.



KNN – Accuracy for Different K Values



KNN – Performance Metrics (Best K = 49)

KNN (K=49) Performance on 70:30 Split

Accuracy : 64.05%
Precision: 65.58%
Recall : 68.44%
F1 Score : 66.98%

Confusion Matrix:

```
[[ 868  602]
 [ 529 1147]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.62	0.59	0.61	1470
1	0.66	0.68	0.67	1676
accuracy			0.64	3146
macro avg	0.64	0.64	0.64	3146
weighted avg	0.64	0.64	0.64	3146

Logistic Regression – Performance Metrics

Logistic Regression Performance on 70:30 Split

Accuracy : 63.45%

Precision: 64.88%

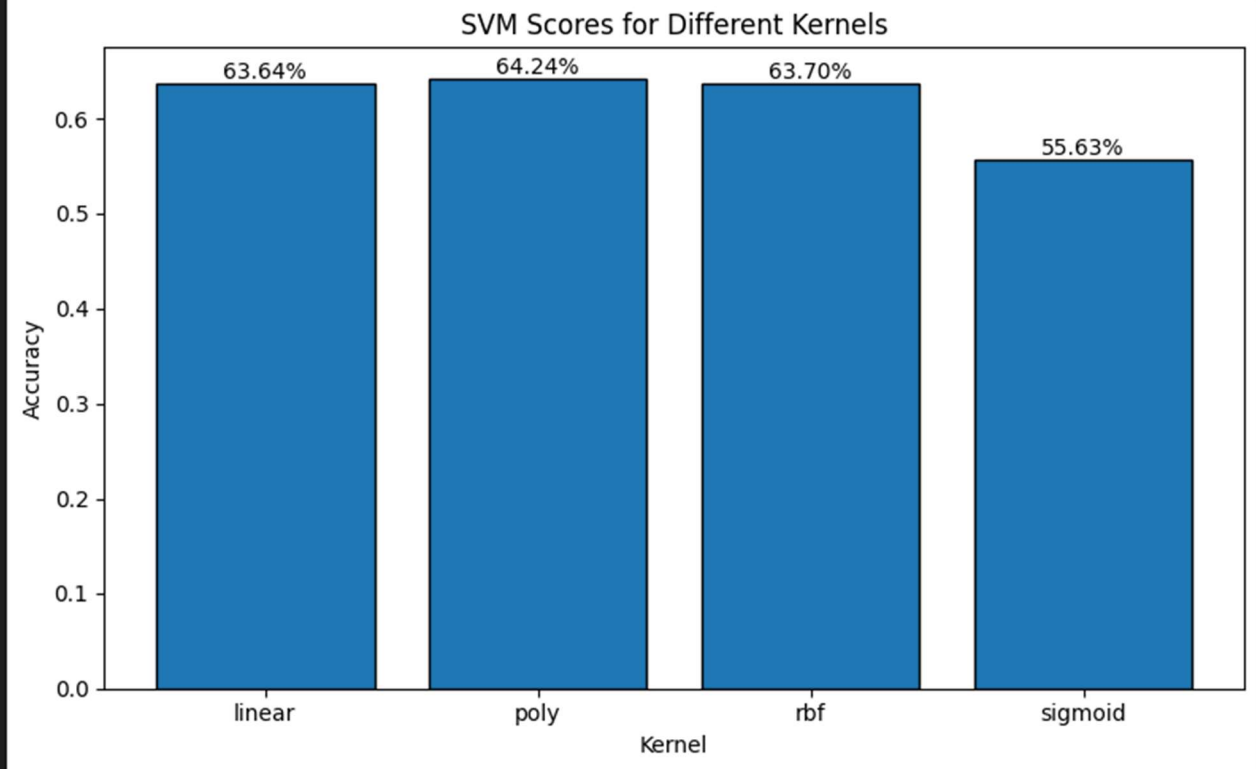
Recall : 68.44%

F1 Score : 66.61%

Classification Report:

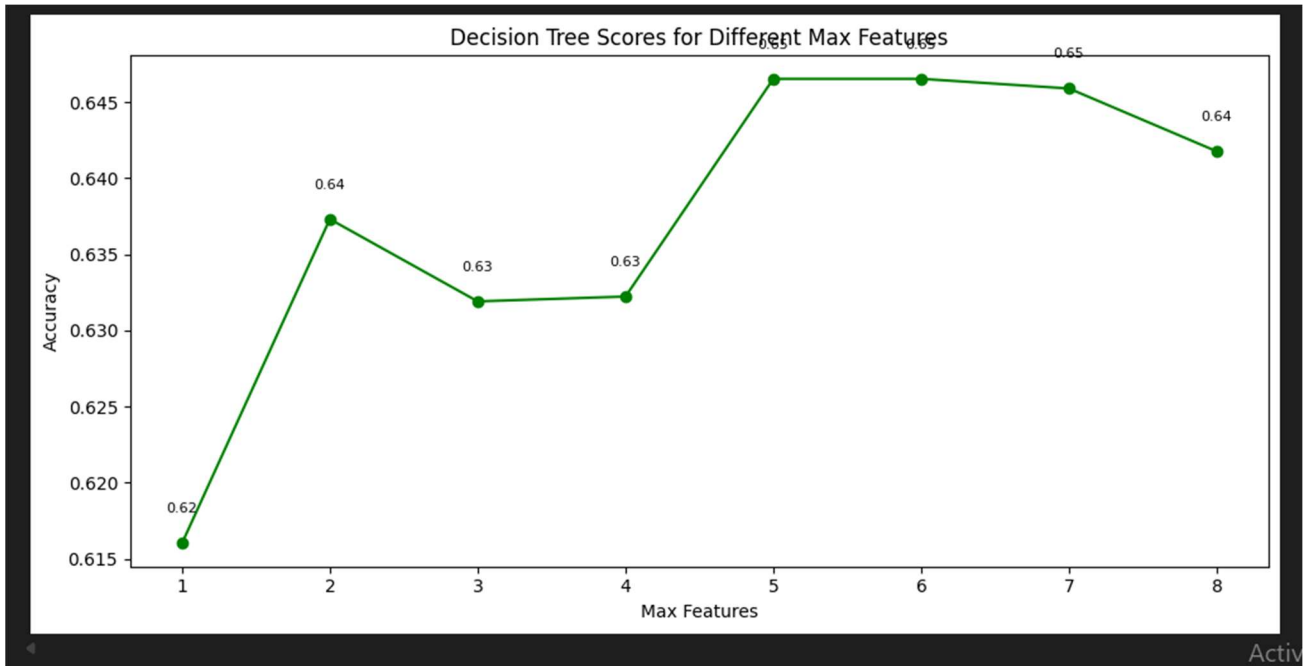
	precision	recall	f1-score	support
0	0.62	0.58	0.60	1470
1	0.65	0.68	0.67	1676
accuracy			0.63	3146
macro avg	0.63	0.63	0.63	3146
weighted avg	0.63	0.63	0.63	3146

SVM – Accuracy for Different Kernels



Best SVM Kernel: poly with accuracy = 64.24%

Decision Tree – Accuracy for Different Max Features

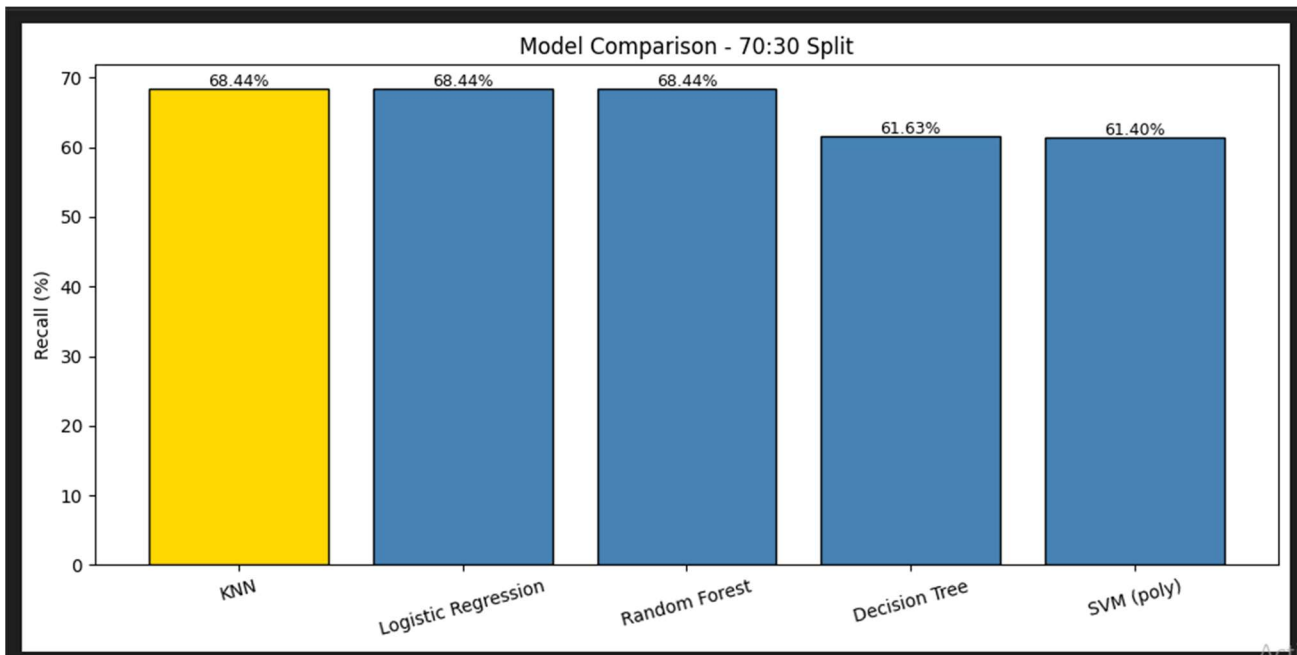


Decision Tree – Performance Metrics

```
===== Final Model Rankings =====
      Model  recall (%)
      KNN    68.436754
Logistic Regression  68.436754
  Random Forest  68.436754
   Decision Tree  61.634845
      SVM (poly)  61.396181
```

Final Model Comparison (70:30 Split)

All five models were trained and tested under identical 70:30 train-test split conditions. The chart below compares their accuracy scores. KNN with K=49 emerged as the best-performing model with 68.43% recall.



===== Final Model Rankings =====

Model	recall (%)
KNN	68.436754
Logistic Regression	68.436754
Random Forest	68.436754
Decision Tree	61.634845
SVM (poly)	61.396181

Model Serialisation & Pickle Test Results

The best model (KNN, K=49), the StandardScaler, and the list of selected features were serialised to disk using Python's pickle module. The saved model was then reloaded and evaluated on the same test set to validate that serialisation preserved model integrity.

The loaded model produced identical results confirming that the serialisation process was successful and the model is ready for deployment.

6. Conclusion and Future Scope

Conclusion

This project successfully developed and evaluated a Heart Disease Prediction System using five machine learning classifiers on a synthetic dataset of 10,485 patient records with 13 clinical features. The key findings are:

- K-Nearest Neighbors (K=49) achieved the highest test recall of 65.83%, making it the best-performing model in this study.
- Sequential Feature Selection reduced the feature space to 8 most discriminative features, improving model efficiency without sacrificing performance.
- StandardScaler normalization of continuous features ensured fair distance-based comparisons in KNN and improved convergence in Logistic Regression and SVM.
- The best model was successfully serialised using Pickle and verified to produce identical results upon reloading, confirming its readiness for deployment.

Future Scope

Several improvements and extensions can be explored in future work:

- Hyperparameter Tuning: Apply Grid Search or Randomised Search cross-validation to optimise hyperparameters for SVM, Random Forest, and Decision Tree models.
- Deep Learning Models: Implement Artificial Neural Networks (ANN) or Convolutional Neural Networks to potentially achieve higher accuracy on larger datasets.
- Real-World Clinical Data: Replace the synthetic dataset with validated real-world clinical datasets such as the UCI Heart Disease Dataset or hospital EHR records.
- Class Imbalance Handling: Apply SMOTE or class-weight adjustments to address any class imbalance and improve minority class recall.
- Web Application Deployment: Build a Flask or Django web application to allow clinicians to input patient data and receive real-time risk predictions.
- Explainability: Integrate SHAP (SHapley Additive exPlanations) or LIME to make model predictions interpretable for medical practitioners.
- Cross-Validation: Implement k-fold cross-validation across all models for more robust and unbiased performance estimation.

7. References:

Books:

1. Let Us Python – Yashavant Kanetkar

Online Resources:

- GeeksforGeeks – <https://www.geeksforgeeks.org/>
- TutorialsPoint – <https://www.tutorialspoint.com/cprogramming/index.htm>
- Stack Overflow – <https://stackoverflow.com>
- <https://matplotlib.org/>
- <https://scikit-learn.org/stable/modules/svm.html>
- <https://www.python.org/>

